California State University, Sacramento
College of Engineering and Computer Science
Computer Science 35: Introduction to Computer Architecture
Spring 2026 – Lab 4: Craps

Part 1. The Program

In this lab you will create a Craps dice game in x86-64 assembly language. Craps is a classic casino dice game with the following rules:

1. **Come-Out Roll: The player rolls two dice (sum = 2–12).**
- If the come-out roll is 7 or 11, the player wins immediately (Natural).
- If the come-out roll is 2, 3, or 12, the player loses immediately (Craps).
- Any other value (4, 5, 6, 8, 9, or 10) becomes the player's "Point."
2. **Point Phase: The player keeps rolling until they hit their Point or a 7.**
- If the player rolls their Point value again, they win.
- If the player rolls a 7, they lose (Seven-Out).
- All other values have no effect; the player rolls again.

Important: This lab uses **registers only** to store all integer values. You will **not** use memory (no .space buffers) for dice values, the point, or game state. The .data section is used only for string constants (.asciz prompts and messages). All integer values live in registers.

Here is an example of the program running:

```
Welcome to Craps!

Rolling the dice...
You rolled a 9. Your point is 9.

Rolling again...
You rolled a 5. Keep rolling!

Rolling again...
You rolled a 9. You hit your point! You win!

---

Welcome to Craps!

Rolling the dice...
You rolled a 7. Natural! You win!

---

Welcome to Craps!
```

```
Rolling the dice...
You rolled a 4. Your point is 4.

Rolling again...
You rolled a 7. Seven-out! You lose.
```

Part 2. Connecting to the Server

If you have not already done so, follow the connection steps from Lab 1 to log in to the server.

- Open MobaXTerm.
- Click Session → SSH.
- Enter the Remote Host:

```
coding1.ecs.csus.edu
```

- Enter your Saclink username and password.

Part 3. The Pseudocode

Below is the complete logic for the program in Gaddis pseudocode. Your job is to translate this into x86-64 assembly language using Intel syntax and the CSC 35 library. Read through the entire pseudocode before you start coding. Pay close attention to the nested If/Else structure inside the two loops.

```
Module Program()

  // --- Declare String Constants ---
  Declare String welcome  = "Welcome to Craps!\n"
  Declare String rollMsg  = "\nRolling the dice...\n"
  Declare String rollAgain= "\nRolling again...\n"
  Declare String rolledMsg= "You rolled a "       // print the number
separately
  Declare String pointMsg = ". Your point is "    // print the point
separately
  Declare String periodNL = ".\n"
  Declare String keepMsg  = ". Keep rolling!\n"
  Declare String winNat   = ". Natural! You win!\n"
  Declare String loseCraps= ". Craps! You lose.\n"
  Declare String winPoint = ". You hit your point! You win!\n"
  Declare String loseSeven= ". Seven-out! You lose.\n"

  // --- Display Welcome ---
  Display welcome

  // --- Come-Out Roll ---
  Display rollMsg
  Set die1 = Random(6) + 1         // 1 to 6
  Set die2 = Random(6) + 1         // 1 to 6
```

```
  Set roll = die1 + die2           // 2 to 12

  Display rolledMsg
  Display roll                      // print the integer

  // --- Evaluate Come-Out Result ---
  If roll == 7 OR roll == 11 Then
    Display winNat
    Call ProgramEnd
  Else If roll == 2 OR roll == 3 OR roll == 12 Then
    Display loseCraps
    Call ProgramEnd
  Else                              // roll is 4,5,6,8,9, or 10 => set
point
    Set point = roll
    Display pointMsg
    Display point                   // print the integer
    Display periodNL
  End If

  // --- Point Phase Loop ---
  Do
    Display rollAgain
    Set die1 = Random(6) + 1
    Set die2 = Random(6) + 1
    Set roll  = die1 + die2

    Display rolledMsg
    Display roll

    If roll == point Then
      Display winPoint
      Call ProgramEnd
    Else If roll == 7 Then
      Display loseSeven
      Call ProgramEnd
    Else
      Display keepMsg
    End If

  While True                        // loop exits via ProgramEnd above

End Module
```

Part 4. Creating Your Source Code

Use the Nano text editor to create your file. Type **nano lab4.asm** and press Enter.

```
nano lab4.asm
```

Begin your file with the following structure. Fill in the .data section with your string constants and translate the pseudocode from Part 3 into the code section.

```
# ------------------------------------------------------------
# Lab 4: Craps
# ------------------------------------------------------------

.intel_syntax noprefix       # Use Intel formatting

.data                        # The DATA section (string constants only)
  # YOUR .asciz STRING LABELS GO HERE

.text                        # The TEXT section (code)
.global Program              # Make the entry point visible

Program:
  # YOUR CODE GOES HERE

  call ProgramEnd            # Terminate the program
```

Press Control+X, then Y, then Enter to save and exit.

Register Usage Guide

Because all integer values must live in registers, plan your register usage before you write a single line. A suggested layout is shown below, but you may organize registers however you like as long as no integer data is stored in memory.

Register	Suggested Use	Notes
`r8`	die1	Value of first die (1–6)
`r9`	die2	Value of second die (1–6)
`r10`	roll	Sum of die1 + die2 (2–12)
`r11`	point	The come-out point value (set once, kept across loop)

Hint: GetRandom destroys the rcx register. Save any values you need across the call by keeping them in callee-saved registers such as r8.

Part 5. Assemble the Program

Assemble your source file into an object file. Type **as -o lab4.o lab4.asm** and press Enter.

```
as -o lab4.o lab4.asm
```

If you have any typos, the assembler will list the errors with line numbers. Open the file with nano and fix them. Common mistakes in this lab include misspelled label names, missing colons after labels (e.g., point_phase: not point_phase), comparing the wrong registers, and forgetting to preserve r12 across GetRandom calls.

Part 6. Link the Objects

Link your object file with the CSC 35 library. Type **ld -o a.out lab4.o csc35.o** and press Enter.

```
ld -o a.out lab4.o csc35.o
```

If you receive “undefined reference” errors, check that your label names match exactly (capitalization matters). If you see “undefined reference to Program”, ensure you have .global Program in your .text section.

Part 7. Execute

Run your program by typing **./a.out** and pressing Enter.

```
./a.out
```

Test all three scenarios: a Natural win (7 or 11 on the come-out), a Craps loss (2, 3, or 12), and the point phase (any other value). Because the dice are random, run the program multiple times until you observe all three outcomes and verify each produces the correct message.

Part 8. Submitting Your Lab

Grading Rubric

Category	Points	Description
Assembly & Execution	2	The program assembles (as), links (ld), and executes without errors.
Dice Simulation	3	Two dice are rolled using GetRandom(6)+1 for each. The sum is computed entirely in registers with no memory usage.
Come-Out Logic	4	Natural (7 or 11) and Craps (2, 3, 12) are detected on the come-out roll and produce the correct win/lose messages. Any other value correctly establishes and displays the point.
Point-Phase Loop	3	The point-phase loop continues until the player rolls their point (win) or a 7 (lose). All other values display the keep-rolling message and loop again.

Output Messages	2	All pseudocode Display statements are correctly translated. Welcome, roll announcement, outcome, and point messages appear at the correct times.
Formatting & Style	1	Code follows standard indentation (instructions indented, labels flush left). Comments are included. File is submitted as .asm.
TOTAL	**15**	

⚠	**This activity may only be submitted using the assembly format used in class:** **(1) x64, (2) Intel-syntax, and (3) the CSC 35 library.** **Using AT&T format will result in a zero. <u>Any work from a prior semester will receive a zero.</u>**

Select the ".asm" file in the MobaXTerm File Explorer pane and download it to the ECS Lab Computer. Once downloaded, open the Canvas Assignment and submit the ".asm" file.

Useful UNIX Commands

Editing

Action	Command	Notes
Edit File	`nano filename`	"Nano" is an easy to use text editor.
Assemble File	`as -o object source`	Don't mix up the object and asm fields. It will destroy your program!
Link File	`ld -o exe object(s)`	Link and create an executable file from one (or more) object files.

Folder Navigation

Action	Command	Notes
Change current folder	`cd foldername`	"Changes Directory"
Go to parent folder	`cd ..`	Think of it as the "back button".
Show current folder	`pwd`	Gives the current file path.

List files	`ls`	Lists the files in current directory.

File Organization

Action	Command	Notes
Create folder	`mkdir foldername`	Folders are called directories in UNIX.
Copy file	`cp oldfile newfile`	Make a copy of an existing file.
Move file	`mv filename foldername`	Moves a file to a destination folder.
Rename file	`mv oldname newname`	Note: same command as “move”.
Delete file	`rm filename`	Remove (delete) a file. There is no undo.